Integrated Development Environments

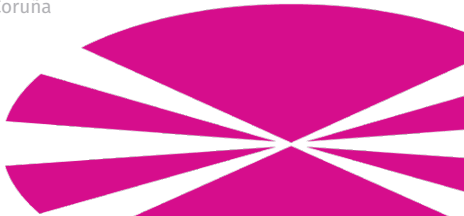
Ferramentas de Desenvolvemento

Javier Parapar López

Grao en Enxeñaría Informática · Facultade de Informática · Universidade da Coruña

Curso 2026/2027

GEI GRADO EN ENXEÑARÍA INFORMÁTICA



Where we are going

1. What an IDE is
2. Visual Studio Code
3. Reading code
4. Writing code faster
5. Debugging
6. Testing
7. The team's editor
8. Version control

SECTION 1

What an IDE is

1

An editor that understands what you wrote

An Integrated Development Environment is a tool that does not treat your program as *text*. It parses it, keeps a model of it in memory, and every feature below is a query over that model.

The four core properties

Language-aware editing — it knows what a class, a method and a scope are.

Project model — source folders, classpath, dependencies.

Integrated compilation — errors as you type, not after a command.

Stepwise execution — a debugger attached to the running program.

An editor that understands what you wrote

An Integrated Development Environment is a tool that does not treat your program as *text*. It parses it, keeps a model of it in memory, and every feature below is a query over that model.

The four core properties

Language-aware editing — it knows what a class, a method and a scope are.

Project model — source folders, classpath, dependencies.

Integrated compilation — errors as you type, not after a command.

Stepwise execution — a debugger attached to the running program.

Built on top of the same model

Wizards · version control (*Lesson 2*) · issue tracking (*Lesson 3*) · **refactoring** · diagramming · test running (*Lesson 9*) · static analysis (*Lesson 6*).

An editor that understands what you wrote

An Integrated Development Environment is a tool that does not treat your program as *text*. It parses it, keeps a model of it in memory, and every feature below is a query over that model.

The four core properties

Language-aware editing — it knows what a class, a method and a scope are.

Project model — source folders, classpath, dependencies.

Integrated compilation — errors as you type, not after a command.

Stepwise execution — a debugger attached to the running program.

Built on top of the same model

Wizards · version control (*Lesson 2*) · issue tracking (*Lesson 3*) · **refactoring** · diagramming · test running (*Lesson 9*) · static analysis (*Lesson 6*).

Keep the distinction in mind all day: a text editor renames a word; an IDE renames a method and gets every call site right, including the ones you forgot.

Why it pays — and what it hides

Why it pays — and what it hides

- **You read more code than you write.** Navigation, call hierarchies and *go to definition* are what make a 200 000-line system approachable at all

Why it pays — and what it hides

- **You read more code than you write.** Navigation, call hierarchies and *go to definition* are what make a 200 000-line system approachable at all
- **The feedback loop shrinks.** Background compilation turns a compile–run–read-the-stack-trace cycle into a red squiggle while you type

Why it pays — and what it hides

- **You read more code than you write.** Navigation, call hierarchies and *go to definition* are what make a 200 000-line system approachable at all
- **The feedback loop shrinks.** Background compilation turns a compile–run–read-the-stack-trace cycle into a red squiggle while you type
- **Mechanical changes become safe.** Rename, extract method, change signature: the IDE applies them across the project and you review the diff

Why it pays — and what it hides

- **You read more code than you write.** Navigation, call hierarchies and *go to definition* are what make a 200 000-line system approachable at all
- **The feedback loop shrinks.** Background compilation turns a compile–run–read-the-stack-trace cycle into a red squiggle while you type
- **Mechanical changes become safe.** Rename, extract method, change signature: the IDE applies them across the project and you review the diff
- **And the debugger replaces guessing** — `System.out.println("llega aquí")` is a debugger with one breakpoint, no variables view and no way to step

But an IDE is also a **comfortable trap**: it builds and runs your project *its own way*. If the build only works inside the IDE, it does not work. That is why *Lesson 4* exists — a project must build from the command line, on a machine that has never seen your editor.

The landscape, and where this class sits

They all offer the same vocabulary — completion, navigation, refactoring, debugging — and differ in how much they assume about your project:

- **Language-first, heavyweight**: IntelliJ IDEA, Eclipse, NetBeans, Xcode, Visual Studio
- **Editor-first, extensible**: Visual Studio Code, Neovim, Emacs
- **AI-first**: Cursor, Windsurf — both are forks of VS Code, so everything in this lesson still applies to them

The landscape, and where this class sits

They all offer the same vocabulary — completion, navigation, refactoring, debugging — and differ in how much they assume about your project:

- **Language-first, heavyweight**: IntelliJ IDEA, Eclipse, NetBeans, Xcode, Visual Studio
- **Editor-first, extensible**: Visual Studio Code, Neovim, Emacs
- **AI-first**: Cursor, Windsurf — both are forks of VS Code, so everything in this lesson still applies to them

Stack Overflow 2025

Of the 26 143 developers who answered:

Visual Studio Code 75.9 %
Visual Studio 29.0 % · Notepad++ 27.4 %
IntelliJ IDEA 27.1 % · Vim 24.3 %
Cursor 17.9 %

The landscape, and where this class sits

They all offer the same vocabulary — completion, navigation, refactoring, debugging — and differ in how much they assume about your project:

- **Language-first, heavyweight**: IntelliJ IDEA, Eclipse, NetBeans, Xcode, Visual Studio
- **Editor-first, extensible**: Visual Studio Code, Neovim, Emacs
- **AI-first**: Cursor, Windsurf — both are forks of VS Code, so everything in this lesson still applies to them

Stack Overflow 2025

Of the 26 143 developers who answered:

Visual Studio Code 75.9 %
Visual Studio 29.0 % · Notepad++ 27.4 %
IntelliJ IDEA 27.1 % · Vim 24.3 %
Cursor 17.9 %

We use VS Code because it is what you are most likely to find, and because it is the same editor whether the project is Java, TypeScript or a Dockerfile. Learn the concepts: the key bindings change every time you change job.

SECTION 2

Visual Studio Code

2

Visual Studio Code

- A **multi-language source-code editor** by Microsoft for Windows, Linux, macOS and the browser. Announced in April 2015, source released under the **MIT licence** in November 2015
- Built on **Electron**: the same application everywhere, at the price of being a web app in a window
- Language support, debugging, testing and version control all arrive as **extensions** — the editor itself knows almost nothing about Java
- Which is the design decision that matters: a small core plus a marketplace. It is why it won, and why it can be a Java IDE on Monday and a YAML editor on Tuesday



Visual Studio Code

code.visualstudio.com

Visual Studio Code

- A **multi-language source-code editor** by Microsoft for Windows, Linux, macOS and the browser. Announced in April 2015, source released under the **MIT licence** in November 2015
- Built on **Electron**: the same application everywhere, at the price of being a web app in a window
- Language support, debugging, testing and version control all arrive as **extensions** — the editor itself knows almost nothing about Java
- Which is the design decision that matters: a small core plus a marketplace. It is why it won, and why it can be a Java IDE on Monday and a YAML editor on Tuesday



code.visualstudio.com

The consequence is worth saying out loud: when you install "Java support" you are installing a separate program that the editor talks to. Half the problems you will hit this term are that program not starting.

Installing it

Free, for macOS, Linux and Windows. On Debian and Ubuntu there are two ways — and only the second one gets you **updates**:

1. the downloaded package

```
sudo apt install ./<file>.deb
```

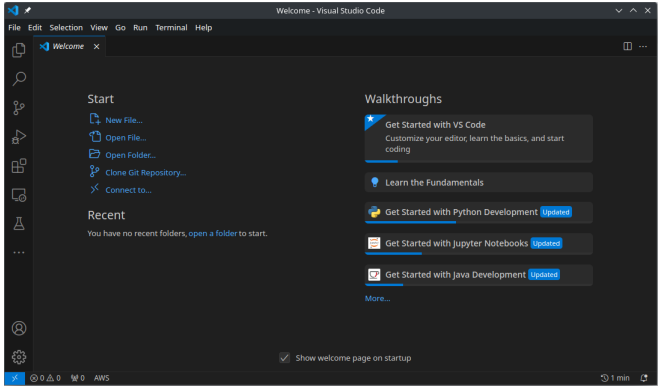
2. the repository — the key first, then the source signed with it, then the install

```
sudo apt-get install wget gpg
wget -qO- https://packages.microsoft.com/keys/microsoft.asc | gpg --dearmor > packages.microsoft.gpg
sudo install -D -o root -g root -m 644 packages.microsoft.gpg /etc/apt/keyrings/packages.microsoft.gpg
echo "deb [arch=amd64,arm64,armhf signed-by=/etc/apt/keyrings/packages.microsoft.gpg] \
   https://packages.microsoft.com/repos/code stable main" | sudo tee /etc/apt/sources.list.d/vscode.list
sudo apt update && sudo apt install code
```

Everything else: `code.visualstudio.com/docs/setup/setup-overview`

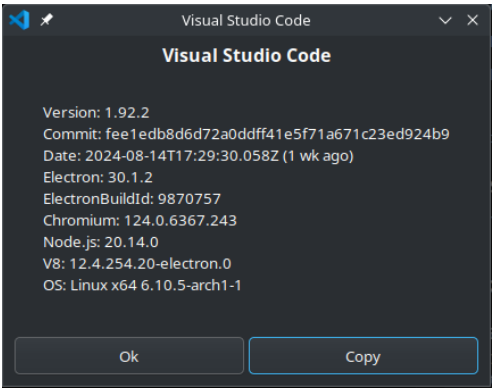
First steps

The editor on first launch — no project, no language, nothing to do yet.



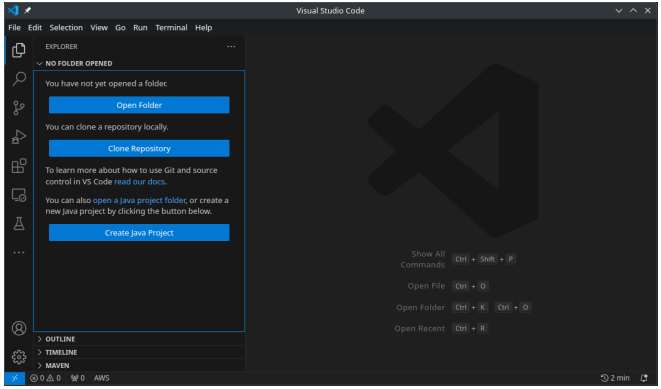
First steps

Version information: the first thing you are asked for in any bug report.



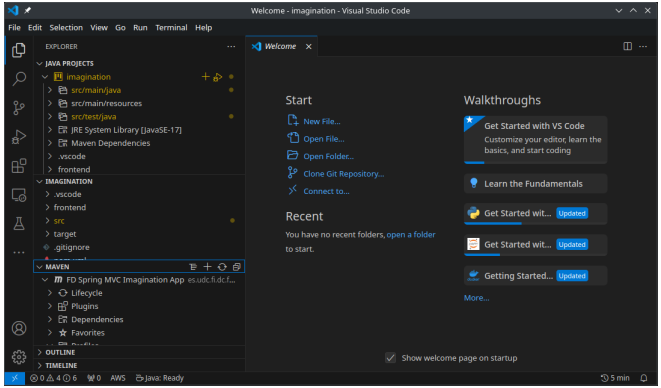
First steps

The layout: activity bar, sidebar, editor group, panel, status bar.



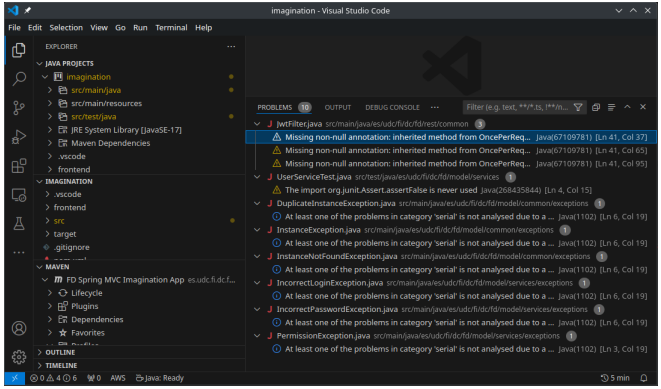
First steps

The Java project view — the *model* of the project, not the folder.



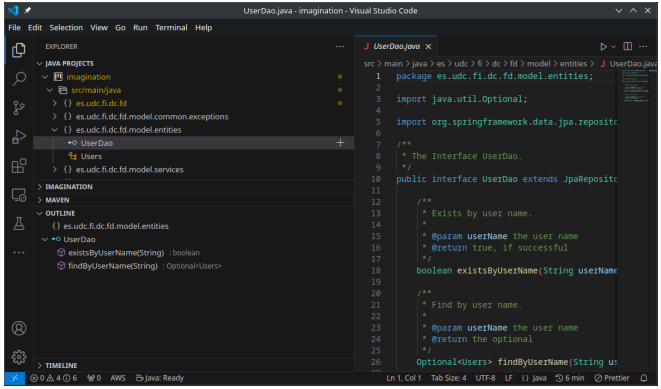
First steps

The Problems view: every diagnostic in the workspace, not just this file.



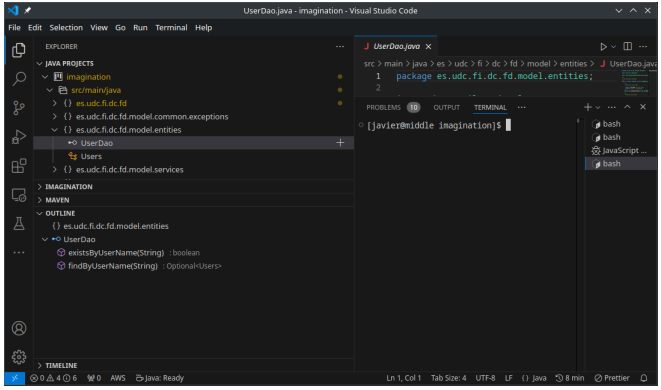
First steps

The Outline: the structure of the file you are looking at.



First steps

The panel is multi-purpose — terminal, output, debug console, problems.



Folders, workspaces, and where settings live

- A workspace is the set of folders opened in one window. Usually one; with **multi-root workspaces** it can be several — the front end and the back end of your project, side by side
- Several folders in one window means **one search**, one terminal and one debug session across all of them
- What it buys you: settings scoped to the project, saved task and debug configurations, restored UI state, and extensions enabled per workspace
- A multi-root workspace lives in a `.code-workspace` file, which you can commit

Folders, workspaces, and where settings live

- A workspace is the set of folders opened in one window. Usually one; with **multi-root workspaces** it can be several — the front end and the back end of your project, side by side
- Several folders in one window means **one search**, one terminal and one debug session across all of them
- What it buys you: settings scoped to the project, saved task and debug configurations, restored UI state, and extensions enabled per workspace
- A multi-root workspace lives in a `.code-workspace` file, which you can commit

Settings have three levels

User — your machine, every project.

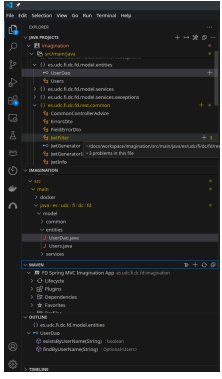
Workspace — this project, *committed with it.*

Folder — one root inside a multi-root workspace.

The middle one is the one that matters to a team, and we come back to it at the end of the lesson.

Views, editors and the panel

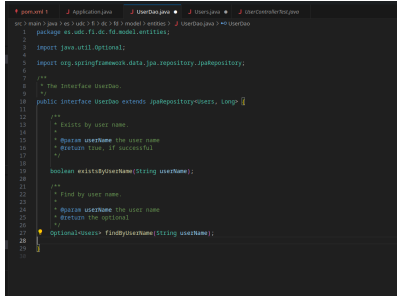
- A view is a container of content that lives in the sidebar or in the panel: tree views (Explorer, Java Projects), welcome views, web views contributed by extensions
- Views carry **view actions** — the little icons in their title bar — and can be dragged between containers, so your layout is yours
- An editor is different: it edits **one element**, usually a file. Editors live in **editor groups**, which is how you get them side by side
- Everything is rearrangeable, which is a trap for a class: the menus I show you may not be where yours are. **Learn the command palette** (`Ctrl+Shift+P`) and you never depend on the layout again



a view

Views, editors and the panel

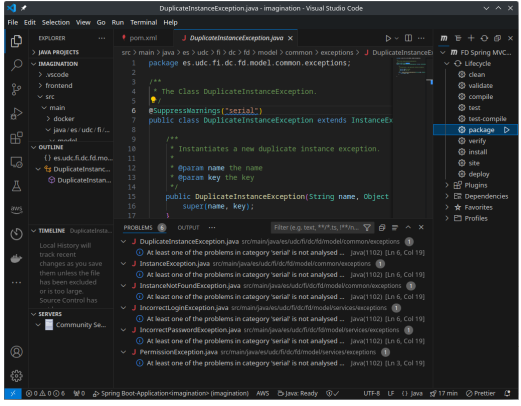
- A view is a container of content that lives in the sidebar or in the panel: tree views (Explorer, Java Projects), welcome views, web views contributed by extensions
- Views carry **view actions** — the little icons in their title bar — and can be dragged between containers, so your layout is yours
- An editor is different: it edits **one element**, usually a file. Editors live in **editor groups**, which is how you get them side by side
- Everything is rearrangeable, which is a trap for a class: the menus I show you may not be where yours are. **Learn the command palette** (`Ctrl+Shift+P`) and you never depend on the layout again



an editor

The four bars

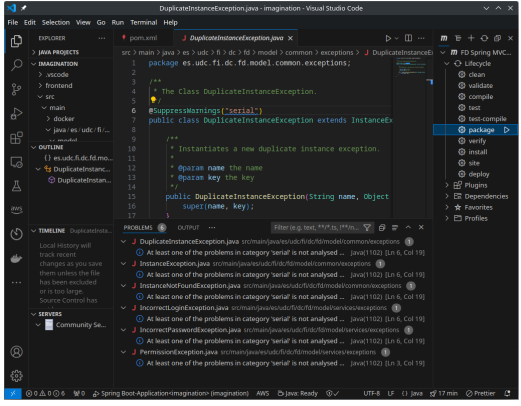
- **Activity bar** (left): the main navigation surface. Extensions contribute view containers here; you can drag them elsewhere
- **Sidebars**, primary and secondary: hold the views of the container you selected
- **Status bar** (bottom): workspace state on the **left** — branch, problems, sync — and context on the **right** — language, encoding, indentation
- Views are added to or removed from a sidebar with the `...` menu in its title bar



the bars

The four bars

- **Activity bar** (left): the main navigation surface. Extensions contribute view containers here; you can drag them elsewhere
- **Sidebars**, primary and secondary: hold the views of the container you selected
- **Status bar** (bottom): workspace state on the **left** — branch, problems, sync — and context on the **right** — language, encoding, indentation
- Views are added to or removed from a sidebar with the `...` menu in its title bar

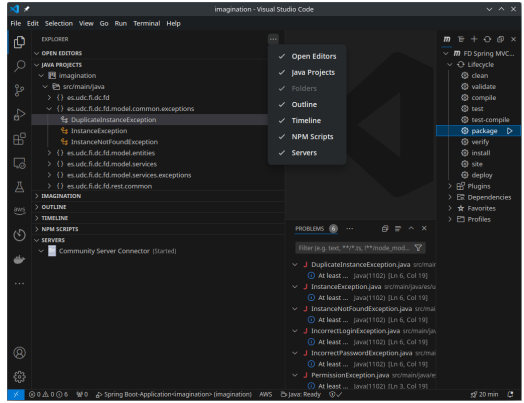


the bars

The four bars

- **Activity bar** (left): the main navigation surface. Extensions contribute view containers here; you can drag them elsewhere
- **Sidebars**, primary and secondary: hold the views of the container you selected
- **Status bar** (bottom): workspace state on the **left** — branch, problems, sync — and context on the **right** — language, encoding, indentation
- Views are added to or removed from a sidebar with the ... menu in its title bar

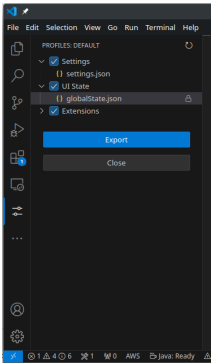
Read the status bar before asking why something fails: it tells you the branch, the language mode and how many problems the workspace has.



adding and removing views

Profiles: one editor, several jobs

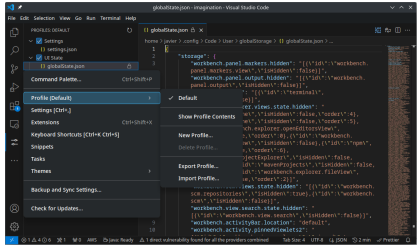
- A profile groups settings, extensions and UI layout, so you can switch between setups — Java here, React there — without uninstalling anything
- Your current configuration is the **Default profile**. New ones are created from it, from an empty one, or from a template: `File > Preferences > Profile > New Profile`, or `Profiles: New Profile` in the palette
- **Partial profiles** override only part of the configuration — say, the settings — and inherit the rest
- Profiles can be **exported and imported**, which is how a team shares a setup, and how you move machines
- The active profile is shown in the title bar and in the gear menu (nothing is shown when it is the default one)



switching profile

Profiles: one editor, several jobs

- A profile groups settings, extensions and UI layout, so you can switch between setups — Java here, React there — without uninstalling anything
- Your current configuration is the **Default profile**. New ones are created from it, from an empty one, or from a template: `File > Preferences > Profile > New Profile`, or `Profiles: New Profile` in the palette
- **Partial profiles** override only part of the configuration — say, the settings — and inherit the rest
- Profiles can be **exported and imported**, which is how a team shares a setup, and how you move machines
- The active profile is shown in the title bar and in the gear menu (nothing is shown when it is the default one)



managing them

What “Java support” actually installs

The **Java General** profile template is the quickest sane start. It pulls in the **Extension Pack for Java**, which is six separate extensions:

EXTENSION	WHAT IT GIVES YOU
Language Support for Java (Red Hat)	the language server: errors, completion, refactoring
Debugger for Java	breakpoints, stepping, variables
Test Runner for Java	run and debug JUnit and TestNG (*Lesson 9*)
Maven for Java	the build, from the editor (*Lesson 4*)
Project Manager for Java	the project model, the Java Projects view
IntelliCode	completion ranked by a model trained on public code

What “Java support” actually installs

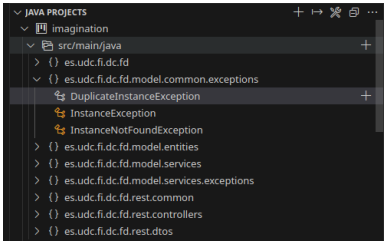
The **Java General** profile template is the quickest sane start. It pulls in the **Extension Pack for Java**, which is six separate extensions:

EXTENSION	WHAT IT GIVES YOU
Language Support for Java (Red Hat)	the language server: errors, completion, refactoring
Debugger for Java	breakpoints, stepping, variables
Test Runner for Java	run and debug JUnit and TestNG (*Lesson 9*)
Maven for Java	the build, from the editor (*Lesson 4*)
Project Manager for Java	the project model, the Java Projects view
IntelliCode	completion ranked by a model trained on public code

Read that table as a map of the term: four of the six rows are lessons of this course. The editor is where they all meet, but none of them *lives* in the editor.

The Java Projects view

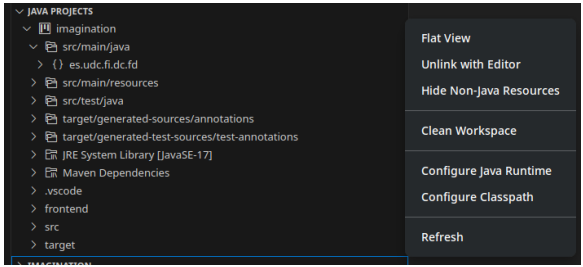
- Manages your Java projects and **their dependencies**: source roots, referenced libraries, JDK
- It sits under the Explorer. Not there? The ... button in the Explorer title bar brings it back
- **Hierarchical** or **flat** view, from the overflow menu — flat is much easier once you have many packages
- The buttons next to each node, and the right-click menu, are where “add dependency” and “new class” live



hierarchical

The Java Projects view

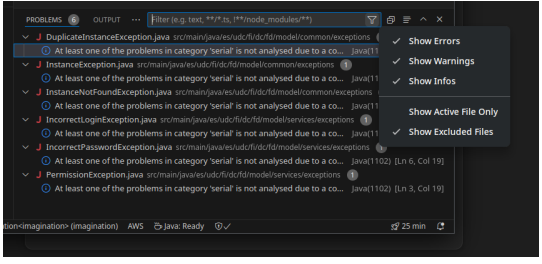
- Manages your Java projects and **their dependencies**: source roots, referenced libraries, JDK
- It sits under the Explorer. Not there? The `...` button in the Explorer title bar brings it back
- **Hierarchical** or **flat** view, from the overflow menu — flat is much easier once you have many packages
- The buttons next to each node, and the right-click menu, are where “add dependency” and “new class” live



flat

The Problems view

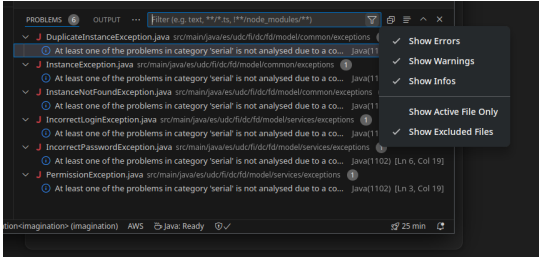
- Every error, warning and hint **in the workspace** — not only in the file you have open. `Ctrl+Shift+M`
- Grouped by file; click to jump to the line
- Filter by **type** (errors / warnings / info), by **text**, by **source** (the tool that produced it) or by excluding paths with globs
- That “source” filter matters later: the same view will show you SonarLint findings in *Lesson 6*



The Problems view

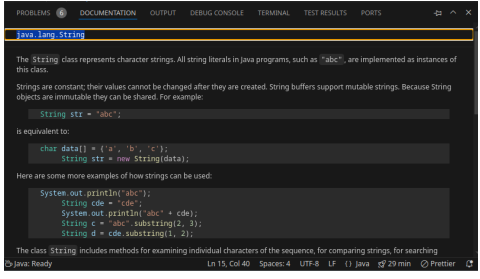
- Every error, warning and hint **in the workspace** — not only in the file you have open. `Ctrl+Shift+M`
- Grouped by file; click to jump to the line
- Filter by **type** (errors / warnings / info), by **text**, by **source** (the tool that produced it) or by excluding paths with globs
- That “source” filter matters later: the same view will show you SonarLint findings in *Lesson 6*

A project with 300 warnings has no warnings: nobody reads them. Getting this view to zero is a real engineering practice, not tidiness.



Documentation, where you are looking

- By default the Javadoc of a symbol appears in a **hover popup** — signature, parameters and comment, without leaving the line you are on
- The **Docs View** extension pins that same content to a panel, so it follows the cursor instead of appearing and vanishing
- Which is an argument for writing Javadoc: it is not decoration, it is what your colleague sees at the moment they are about to call your method



SECTION 3

Reading code

3

Moving through code without scrolling

Scrolling is how you read a file. These are how you read a **system** — and every one of them is a query over the model:

WHAT YOU WANT	HOW	KEY
Where is this defined?	Go to Definition	`F12`
…without losing my place	Peek Definition	`Ctrl+Shift+F10`
Which class is this really?	Go to Super Implementation	hover, then click
Who calls this method?	Call Hierarchy	right-click, *Peek*
A symbol, anywhere in the project	Workspace symbol search	`Ctrl+T`
A symbol, in this file	Quick open, then `@`	`Ctrl+P`
The shape of this file	Outline view	sidebar

Moving through code without scrolling

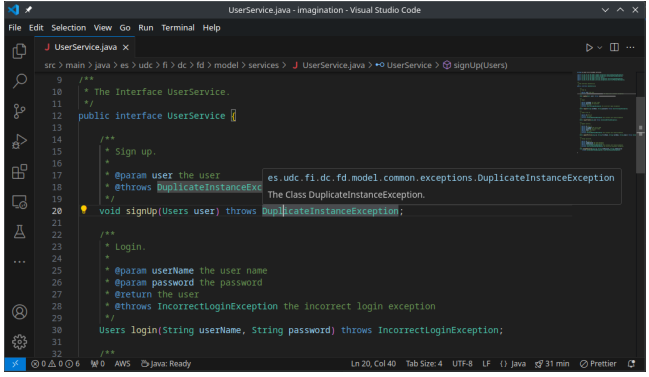
Scrolling is how you read a file. These are how you read a **system** — and every one of them is a query over the model:

WHAT YOU WANT	HOW	KEY
Where is this defined?	Go to Definition	`F12`
…without losing my place	Peek Definition	`Ctrl+Shift+F10`
Which class is this really?	Go to Super Implementation	hover, then click
Who calls this method?	Call Hierarchy	right-click, *Peek*
A symbol, anywhere in the project	Workspace symbol search	`Ctrl+T`
A symbol, in this file	Quick open, then `@`	`Ctrl+P`
The shape of this file	Outline view	sidebar

Peek is the one to build a habit around: it opens the definition *inside* the current editor, so you answer the question and carry on without ten tabs and no idea how you got there.

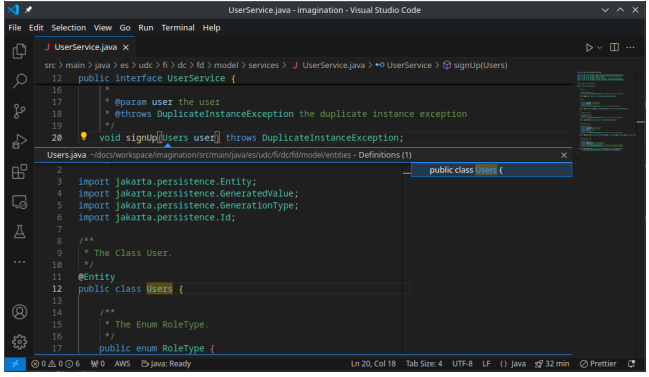
Navigation, in practice

The Java editor: breadcrumbs on top, outline on the side, diagnostics inline.



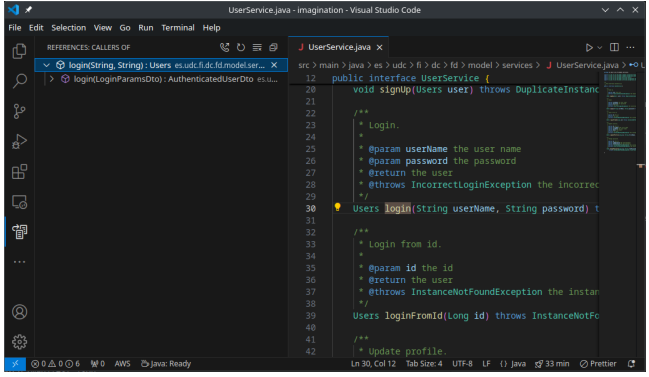
Navigation, in practice

Peek Definition — the definition, inside the file you were reading.



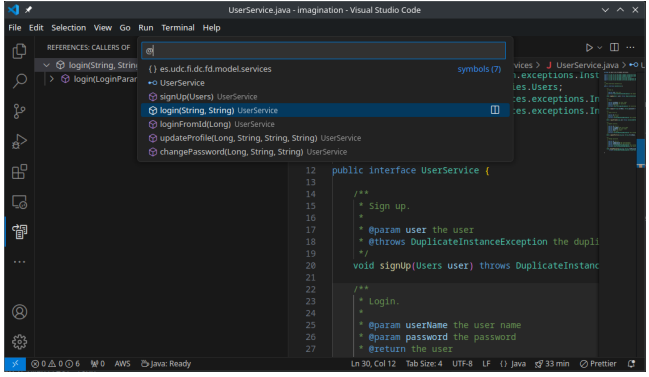
Navigation, in practice

Call Hierarchy — who calls this, and who calls them.



Navigation, in practice

Workspace symbols (`Ctrl+T`) — the whole project, by name.



When the framework is the structure

In a Spring Boot project, half the structure is not in the class hierarchy: it is in the annotations. The **Spring Boot Tools** extension teaches `Ctrl+T` to search that structure too:

PREFIX	WHAT IT LISTS
`@/`	all request mappings: path, HTTP method, where it is defined
`@+`	all beans: name, type, source
`@>`	all functions and their prototype implementations
`@`	every Spring annotation in the project

When the framework is the structure

In a Spring Boot project, half the structure is not in the class hierarchy: it is in the annotations. The **Spring Boot Tools** extension teaches `Ctrl+T` to search that structure too:

PREFIX	WHAT IT LISTS
`@/`	all request mappings: path, HTTP method, where it is defined
`@+`	all beans: name, type, source
`@>`	all functions and their prototype implementations
`@`	every Spring annotation in the project

Worth ten minutes the first time you open somebody else's Spring project: `@/` answers "what does this service actually expose?" faster than any documentation you will find.

SECTION 4

Writing code faster



Four things that write code for you

Four things that write code for you

- **IntelliSense** — completion with signatures and documentation, driven by the language server, not by the words already in the file

Four things that write code for you

- **IntelliSense** — completion with signatures and documentation, driven by the language server, not by the words already in the file
- **Snippets** — templates with tab stops for the shapes you type twenty times a day

Four things that write code for you

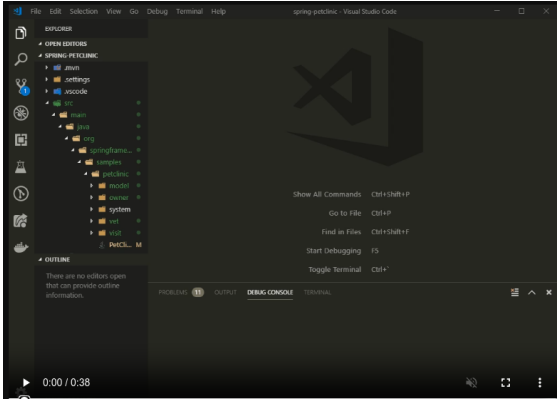
- **IntelliSense** — completion with signatures and documentation, driven by the language server, not by the words already in the file
- **Snippets** — templates with tab stops for the shapes you type twenty times a day
- **Code actions** — generate getters and setters, generate tests, organise imports: edits derived from the model

Four things that write code for you

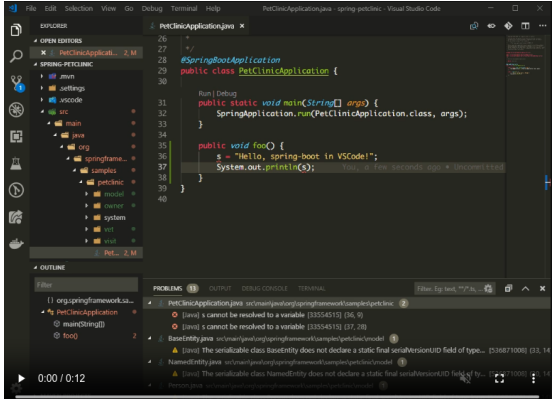
- **IntelliSense** — completion with signatures and documentation, driven by the language server, not by the words already in the file
- **Snippets** — templates with tab stops for the shapes you type twenty times a day
- **Code actions** — generate getters and setters, generate tests, organise imports: edits derived from the model
- **Quick fixes** (`Ctrl+.`) — the same mechanism, but offered *where the problem is*: unresolved import, missing method, unhandled exception

The habit worth forming: when you see a red squiggle, press `Ctrl+.` before you start typing. The editor has already read the error message, and it is usually right.

Editing, live



Quick fixes, live



The snippets worth memorising

There are dozens. These are the ones that pay for themselves in a week — type the prefix, press `Tab`:

PREFIX	GIVES YOU
`main`, `psvm`	the main method
`sout`, `sysout`	`System.out.println()`
`serr`	`System.err.println()`
`fori`	indexed `for` over an array
`iter`, `foreach`	enhanced `for`
`try_catch`	`try`/`catch` block
`try_resources`	try-with-resources
`ctor`	public constructor

The snippets worth memorising

There are dozens. These are the ones that pay for themselves in a week — type the prefix, press `Tab`:

PREFIX	GIVES YOU
`main`, `psvm`	the main method
`sout`, `sysout`	`System.out.println()`
`serr`	`System.err.println()`
`fori`	indexed `for` over an array
`iter`, `foreach`	enhanced `for`
`try_catch`	`try`/`catch` block
`try_resources`	try-with-resources
`ctor`	public constructor

Postfix snippets

Type the expression **first**, then the suffix — `items.for`, `name.null`, `result.var`, `x.sysout`. It matches how you think: you know *what* before you know what to *do* with it.

The snippets worth memorising

There are dozens. These are the ones that pay for themselves in a week — type the prefix, press `Tab`:

PREFIX	GIVES YOU
`main`, `psvm`	the main method
`sout`, `sysout`	`System.out.println()`
`serr`	`System.err.println()`
`fori`	indexed `for` over an array
`iter`, `foreach`	enhanced `for`
`try_catch`	`try`/`catch` block
`try_resources`	try-with-resources
`ctor`	public constructor

Postfix snippets

Type the expression **first**, then the suffix — `items.for`, `name.null`, `result.var`, `x.sysout`. It matches how you think: you know *what* before you know what to *do* with it.

The full tables — plain, postfix, and the modifier ones like `private_method` — are in the appendix of these slides, and the editor itself lists them: start typing and read the popup.

Refactoring: changing the code, not the behaviour

Refactoring is a system-wide change to the structure of a program that leaves what it does unchanged. The IDE can do it **safely** for the same reason it can navigate: it has the model, so it knows what is a reference and what is merely the same word.

- Select the element, then `Ctrl+Shift+R` or right-click **Refactor…**
- **Rename** is F2, and it is the one you will use every day
- Everything else lives in the same menu, grouped by what it does — next slide

Refactoring: changing the code, not the behaviour

Refactoring is a system-wide change to the structure of a program that leaves what it does unchanged. The IDE can do it **safely** for the same reason it can navigate: it has the model, so it knows what is a reference and what is merely the same word.

- Select the element, then `Ctrl+Shift+R` or right-click **Refactor…**
- **Rename** is F2, and it is the one you will use every day
- Everything else lives in the same menu, grouped by what it does — next slide

Refactor *then* commit

Never mix a refactoring with a change of behaviour in the same commit: the diff becomes unreviewable and nobody can tell which line changed what. One commit that renames, one commit that fixes. *Lesson 2* again.

Refactoring: changing the code, not the behaviour

Refactoring is a system-wide change to the structure of a program that leaves what it does unchanged. The IDE can do it **safely** for the same reason it can navigate: it has the model, so it knows what is a reference and what is merely the same word.

- Select the element, then `Ctrl+Shift+R` or right-click **Refactor…**
- **Rename** is `F2`, and it is the one you will use every day
- Everything else lives in the same menu, grouped by what it does — next slide

Refactor *then* commit

Never mix a refactoring with a change of behaviour in the same commit: the diff becomes unreviewable and nobody can tell which line changed what. One commit that renames, one commit that fixes. *Lesson 2* again.

And the safety net underneath all of this is the test suite of Lesson 9. Refactoring without tests is just editing with confidence.

The refactoring catalogue

FAMILY	ACTIONS
Give it a name	extract to *local variable*, to *field*, to *constant*, to *method*
Take the name away	inline local variable, inline constant, inline method
Move it	class to another package · method to another class · inner class to a new file
Rename it	`F2` — updates every reference, in every file
Change the form	anonymous class ↔ lambda · `for` → enhanced `for` · static import
Flip it	invert condition · invert boolean variable

The refactoring catalogue

FAMILY	ACTIONS
Give it a name	extract to *local variable*, to *field*, to *constant*, to *method*
Take the name away	inline local variable, inline constant, inline method
Move it	class to another package · method to another class · inner class to a new file
Rename it	`F2` — updates every reference, in every file
Change the form	anonymous class ↔ lambda · `for` → enhanced `for` · static import
Flip it	invert condition · invert boolean variable

The first two rows are one pair of scissors: **extract** when a piece of code deserves a name, **inline** when the name is not earning its keep. Most day-to-day refactoring is those two, applied with judgement.

The refactoring catalogue

FAMILY	ACTIONS
Give it a name	extract to *local variable*, to *field*, to *constant*, to *method*
Take the name away	inline local variable, inline constant, inline method
Move it	class to another package · method to another class · inner class to a new file
Rename it	`F2` — updates every reference, in every file
Change the form	anonymous class ↔ lambda · `for` → enhanced `for` · static import
Flip it	invert condition · invert boolean variable

The first two rows are one pair of scissors: **extract** when a piece of code deserves a name, **inline** when the name is not earning its keep. Most day-to-day refactoring is those two, applied with judgement.

Extract method is the one that changes how you write: when you cannot name the piece you extracted, the code is telling you that it does more than one thing.

Source actions: the code you should not type

- Generate **constructors** from the fields
- Generate **getters and setters**
- Generate **`hashCode()` and `equals()`** — from the fields *you* choose, which is the interesting part
- Generate **`toString()`**
- **Override / implement** methods from a superclass or interface
- Generate **delegate methods**
- **Organise imports**, and make what can be `final`, `final`

Source actions: the code you should not type

- Generate **constructors** from the fields
- Generate **getters and setters**
- Generate **`hashCode()` and `equals()`** — from the fields *you* choose, which is the interesting part
- Generate **`toString()`**
- **Override / implement** methods from a superclass or interface
- Generate **delegate methods**
- **Organise imports**, and make what can be `final`, `final`

Why it matters that it is generated

Hand-written `equals()` and `hashCode()` are a classic source of bugs that only show up when an object goes into a `HashMap`. Generated ones are consistent with each other by construction.

And when a field is added later, you **regenerate** — which is only safe because the generated block is recognisable.

Source actions: the code you should not type

- Generate **constructors** from the fields
- Generate **getters and setters**
- Generate **`hashCode()` and `equals()`** — from the fields *you* choose, which is the interesting part
- Generate **`toString()`**
- **Override / implement** methods from a superclass or interface
- Generate **delegate methods**
- **Organise imports**, and make what can be `final`, `final`

Why it matters that it is generated

Hand-written `equals()` and `hashCode()` are a classic source of bugs that only show up when an object goes into a `HashMap`. Generated ones are consistent with each other by construction.

And when a field is added later, you **regenerate** — which is only safe because the generated block is recognisable.

Same rule as before: generate, then read what was generated. It is your code once it is committed, whoever typed it.

Visual Studio Code

Keyboard shortcuts for Linux

General

Ctrl+Shift+P, F1	Show Command Palette
Ctrl+P	Quick Open, Go to File...
Ctrl+Shift+N	New window/instance
Ctrl+W	Close window/instance
Ctrl+,	User Settings
Ctrl+K Ctrl+S	Keyboard Shortcuts

Basic editing

Ctrl+X	Cut line (empty selection)
Ctrl+C	Copy line (empty selection)
Alt+ ↓ / ↑	Move line down/up
Ctrl+Shift+K	Delete line
Ctrl+Enter / Ctrl+Shift+Enter	Insert line below/ above
Ctrl+Shift+\	Jump to matching bracket
Ctrl+] / Ctrl+[	Indent/Outdent line
Home / End	Go to beginning/end of line
Ctrl+ Home / End	Go to beginning/end of file
Ctrl+ ↑ / ↓	Scroll line up/down
Alt+ PgUp / PgDn	Scroll page up/down
Ctrl+Shift+ [/]	Fold/unfold region
Ctrl+K Ctrl+ [/]	Fold/unfold all subregions
Ctrl+K Ctrl+0 / Ctrl+K Ctrl+J	Fold/Unfold all regions
Ctrl+K Ctrl+C	Add line comment
Ctrl+K Ctrl+U	Remove line comment
Ctrl+/	Toggle line comment
Ctrl+Shift+A	Toggle block comment
Alt+Z	Toggle word wrap

Rich languages editing

Ctrl+Space, Ctrl+I	Trigger suggestion
Ctrl+Shift+Space	Trigger parameter hints
Ctrl+Shift+I	Format document
Ctrl+K Ctrl+F	Format selection
F12	Go to Definition
Ctrl+Shift+F10	Peek Definition
Ctrl+K F12	Open Definition to the side
Ctrl+.	Quick Fix
Shift+F12	Show References
F2	Rename Symbol
Ctrl+K Ctrl+X	Trim trailing whitespace
Ctrl+K M	Change file language

Multi-cursor and selection

Alt+Click	Insert cursor*
Shift+Alt+ ↑ / ↓	Insert cursor above/below
Ctrl+U	Undo last cursor operation
Shift+Alt+I	Insert cursor at end of each line selected
Ctrl+L	Select current line
Ctrl+Shift+L	Select all occurrences of current selection
Ctrl+F2	Select all occurrences of current word
Shift+Alt + →	Expand selection
Shift+Alt + ←	Shrink selection
Shift+Alt + drag mouse	Column (box) selection

Display

F11	Toggle full screen
Shift+Alt+0	Toggle editor layout (horizontal/vertical)
Ctrl+ = / -	Zoom in/out
Ctrl+B	Toggle Sidebar visibility
Ctrl+Shift+E	Show Explorer / Toggle focus
Ctrl+Shift+F	Show Search
Ctrl+Shift+G	Show Source Control
Ctrl+Shift+D	Show Debug
Ctrl+Shift+X	Show Extensions
Ctrl+Shift+H	Replace in files
Ctrl+Shift+J	Toggle Search details
Ctrl+Shift+C	Open new command prompt/terminal
Ctrl+K Ctrl+H	Show Output panel
Ctrl+Shift+V	Open Markdown preview
Ctrl+K V	Open Markdown preview to the side
Ctrl+K Z	Zen Mode (Esc Esc to exit)

Search and replace

Ctrl+F	Find
Ctrl+H	Replace
F3 / Shift+F3	Find next/previous
Alt+Enter	Select all occurrences of Find match
Ctrl+D	Add selection to next Find match
Ctrl+K Ctrl+D	Move last selection to next Find match

Navigation

Ctrl+T	Show all Symbols
Ctrl+G	Go to Line...
Ctrl+P	Go to File...
Ctrl+Shift+O	Go to Symbol...
Ctrl+Shift+M	Show Problems panel
F8	Go to next error or warning
Shift+F8	Go to previous error or warning
Ctrl+Shift+Tab	Navigate editor group history
Ctrl+Alt+-	Go back
Ctrl+Shift+-	Go forward
Ctrl+M	Toggle Tab moves focus

Editor management

Ctrl+W	Close editor
Ctrl+K F	Close folder
Ctrl+\	Split editor
Ctrl+ 1 / 2 / 3	Focus into 1st, 2nd, 3rd editor group
Ctrl+K Ctrl + ←	Focus into previous editor group
Ctrl+K Ctrl + →	Focus into next editor group
Ctrl+Shift+PgUp	Move editor left
Ctrl+Shift+PgDn	Move editor right
Ctrl+K ←	Move active editor group left/up
Ctrl+K →	Move active editor group right/down

File management

Ctrl+N	New File
Ctrl+O	Open File...
Ctrl+S	Save
Ctrl+Shift+S	Save As...
Ctrl+W	Close
Ctrl+K Ctrl+W	Close All
Ctrl+Shift+T	Reopen closed editor
Ctrl+K Enter	Keep preview mode editor open
Ctrl+Tab	Open next
Ctrl+Shift+Tab	Open previous
Ctrl+K P	Copy path of active file
Ctrl+K R	Reveal active file in Explorer
Ctrl+K O	Show active file in new window/instance

Debug

F9	Toggle breakpoint
F5	Start / Continue
F11 / Shift+F11	Step into/out
F10	Step over
Shift+F5	Stop
Ctrl+K Ctrl+I	Show hover

Integrated terminal

Ctrl+`	Show integrated terminal
Ctrl+Shift+`	Create new terminal
Ctrl+Shift+C	Copy selection
Ctrl+Shift+V	Paste into active terminal
Ctrl+Shift+ ↑ / ↓	Scroll up/down
Shift+ PgUp / PgDn	Scroll page up/down
Shift+ Home / End	Scroll to top/bottom

* The Alt+Click gesture may not work on some Linux distributions. You can change the modifier key for the Insert cursor command to Ctrl+Click with the "editor.multiCursorModifier" setting.

SECTION 5

Debugging

5

The six stages of debugging

1. That can’t happen
2. That doesn’t happen on my machine
3. That shouldn’t happen
4. Why does that happen?
5. Oh, I see
6. How did that ever work?

The six stages of debugging

1. That can’t happen
2. That doesn’t happen on my machine
3. That shouldn’t happen
4. Why does that happen?
5. Oh, I see
6. How did that ever work?

It is a joke, and it is also an accurate description of a method. Stages 1 to 3 are **hypotheses you have not tested**; a debugger is how you get to stage 4 in minutes instead of an afternoon — by **observing** the program rather than arguing with it.

Five kinds of breakpoint

KIND	STOPS WHEN
Line	execution reaches this line
Conditional	it reaches the line *and* your expression is true
Data	a variable changes value — set it during a session, from the variables view
Logpoint	never: it **prints** a message and carries on
Triggered	it reaches this line, but only after another breakpoint was hit first

Five kinds of breakpoint

KIND	STOPS WHEN
Line	execution reaches this line
Conditional	it reaches the line *and* your expression is true
Data	a variable changes value — set it during a session, from the variables view
Logpoint	never: it **prints** a message and carries on
Triggered	it reaches this line, but only after another breakpoint was hit first

The last three are the ones nobody uses and everybody needs. A **logpoint** is the `println` you were about to add — without editing the file, without recompiling, and without the risk of committing it by accident. A **conditional** breakpoint is how you debug the 4 000th iteration.

Five kinds of breakpoint

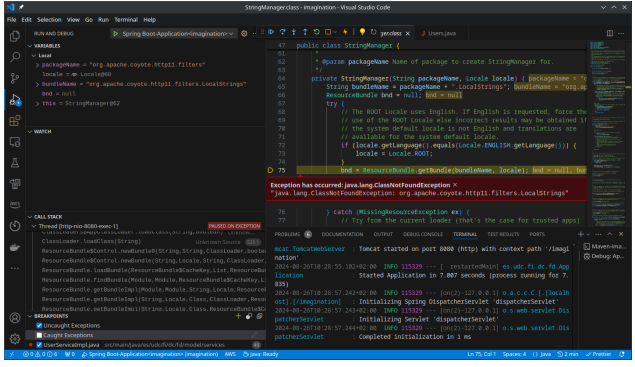
KIND	STOPS WHEN
Line	execution reaches this line
Conditional	it reaches the line *and* your expression is true
Data	a variable changes value — set it during a session, from the variables view
Logpoint	never: it **prints** a message and carries on
Triggered	it reaches this line, but only after another breakpoint was hit first

The last three are the ones nobody uses and everybody needs. A **logpoint** is the `println` you were about to add — without editing the file, without recompiling, and without the risk of committing it by accident. A **conditional** breakpoint is how you debug the 4 000th iteration.

Data breakpoints answer the hardest question in debugging — who changed this field? — which stepping through code almost never answers.

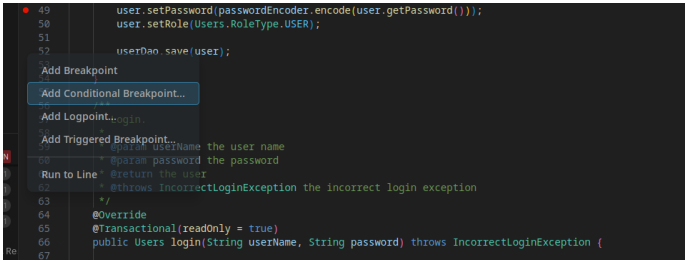
Breakpoints, in practice

The Run and Debug view: breakpoints, variables, watch, call stack.



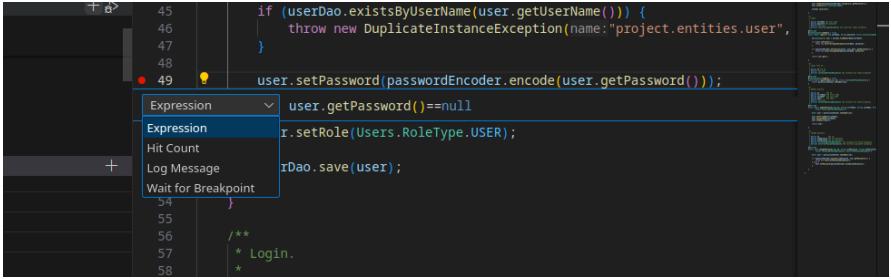
Breakpoints, in practice

Setting a line breakpoint — click the gutter.



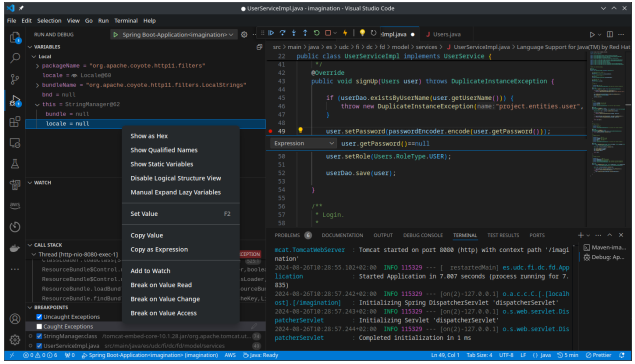
Breakpoints, in practice

A **conditional** breakpoint: an expression, evaluated in scope.



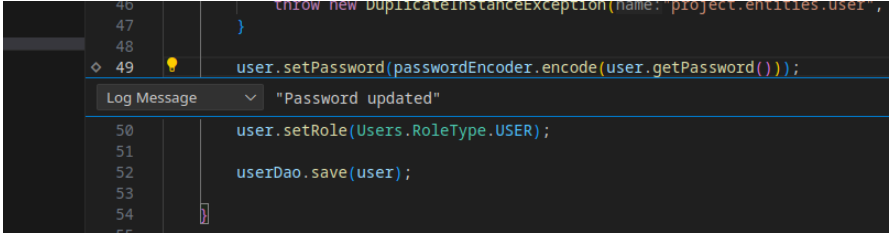
Breakpoints, in practice

A **data** breakpoint, set from the variables view during a session.



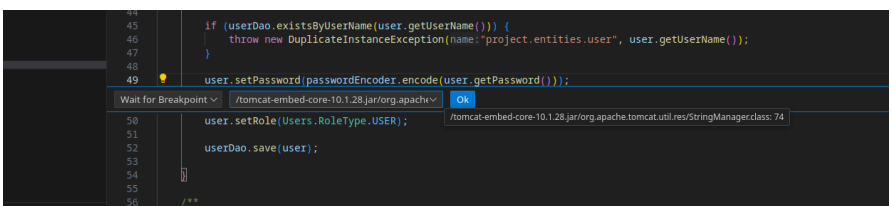
Breakpoints, in practice

A **logpoint**: a message, curly braces to interpolate expressions.



Breakpoints, in practice

A **triggered** breakpoint: armed only after another one is hit.



Four keys, and what they really do

KEY	ACTION
`F5`	**continue** — run until the next breakpoint
`F10`	**step over** — execute the call, do not enter it
`F11`	**step into** — enter the call
`Shift+F11`	**step out** — finish this method, stop at the caller

The mistake everybody makes is living in `F11`. You end up inside the standard library, six frames deep, having lost the thread. Step **over** by default; step **into** only when you suspect *that* call.



Four keys, and what they really do

KEY	ACTION
`F5`	**continue** — run until the next breakpoint
`F10`	**step over** — execute the call, do not enter it
`F11`	**step into** — enter the call
`Shift+F11`	**step out** — finish this method, stop at the caller

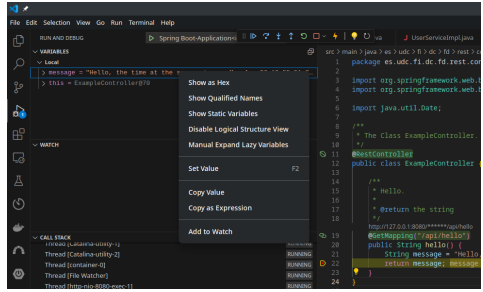


The mistake everybody makes is living in `F11`. You end up inside the standard library, six frames deep, having lost the thread. Step **over** by default; step **into** only when you suspect *that* call.

And `Shift+F11` is the escape hatch: you went in, it was not there, get out without stepping through the rest of it.

What to look at while it is stopped

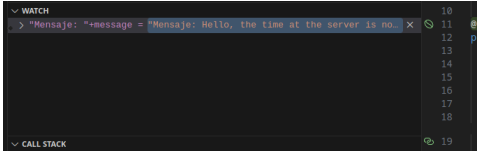
- **Variables** — fields and locals in the current frame, and you can **change a value in place** to test a hypothesis without editing code
- **Watch** — expressions you want evaluated at every stop, not just variables: `list.size()`, `a.equals(b)`
- **Call stack** — how you got here. Click a frame to inspect *its* variables; **Restart Frame** re-runs one frame without restarting the session
- **Debug console** — a live interpreter in the current scope. The fastest way to answer “what is this object, really”



Variables

What to look at while it is stopped

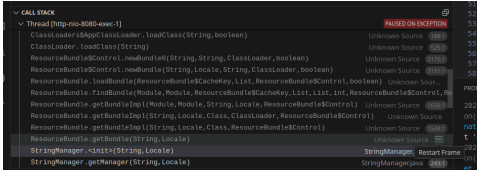
- **Variables** — fields and locals in the current frame, and you can **change a value in place** to test a hypothesis without editing code
- **Watch** — expressions you want evaluated at every stop, not just variables: `list.size()`, `a.equals(b)`
- **Call stack** — how you got here. Click a frame to inspect *its* variables; **Restart Frame** re-runs one frame without restarting the session
- **Debug console** — a live interpreter in the current scope. The fastest way to answer “what is this object, really”



Watch

What to look at while it is stopped

- **Variables** — fields and locals in the current frame, and you can **change a value in place** to test a hypothesis without editing code
- **Watch** — expressions you want evaluated at every stop, not just variables: `list.size()`, `a.equals(b)`
- **Call stack** — how you got here. Click a frame to inspect *its* variables; **Restart Frame** re-runs one frame without restarting the session
- **Debug console** — a live interpreter in the current scope. The fastest way to answer “what is this object, really”



Call stack

What to look at while it is stopped

- **Variables** — fields and locals in the current frame, and you can **change a value in place** to test a hypothesis without editing code
- **Watch** — expressions you want evaluated at every stop, not just variables: `list.size()`, `a.equals(b)`
- **Call stack** — how you got here. Click a frame to inspect *its* variables; **Restart Frame** re-runs one frame without restarting the session
- **Debug console** — a live interpreter in the current scope. The fastest way to answer “what is this object, really”



Debug console

Hot Code Replace

Hot Code Replace sends a changed class into a **running** JVM: you fix and continue without restarting. It exists because restarting is not free — you lose the state that took twenty minutes to reproduce.

What it is good for

Trial and error inside a session: change a method body, save, and the running program picks it up.

Hot Code Replace

Hot Code Replace sends a changed class into a **running** JVM: you fix and continue without restarting. It exists because restarting is not free — you lose the state that took twenty minutes to reproduce.

What it is good for

Trial and error inside a session: change a method body, save, and the running program picks it up.

What it cannot do

The JVM accepts new **method bodies**. Adding or removing methods or fields, or changing a signature, usually forces a restart — and when in doubt, restart.

Hot Code Replace

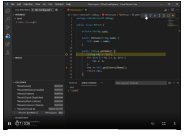
Hot Code Replace sends a changed class into a **running** JVM: you fix and continue without restarting. It exists because restarting is not free — you lose the state that took twenty minutes to reproduce.

What it is good for

Trial and error inside a session: change a method body, save, and the running program picks it up.

What it cannot do

The JVM accepts new **method bodies**. Adding or removing methods or fields, or changing a signature, usually forces a restart — and when in doubt, restart.



The same debugger, somewhere else

The debugger does not have to run where you are. Three cases you will meet this term:

The same debugger, somewhere else

The debugger does not have to run where you are. Three cases you will meet this term:

- **An application server.** VS Code embeds none. You start the server yourself and **attach** — or use the **Spring Boot Dashboard**, or Red Hat's **Community Server Connectors** for Tomcat, Karaf and friends

The same debugger, somewhere else

The debugger does not have to run where you are. Three cases you will meet this term:

- **An application server.** VS Code embeds none. You start the server yourself and **attach** — or use the **Spring Boot Dashboard**, or Red Hat's **Community Server Connectors** for Tomcat, Karaf and friends
- **A browser.** Create a `launch.json` for Chrome, point it at your dev server's port, press `F5`: breakpoints in your React code, hit in the browser. Vite's hot module replacement keeps the application's state while you edit

The same debugger, somewhere else

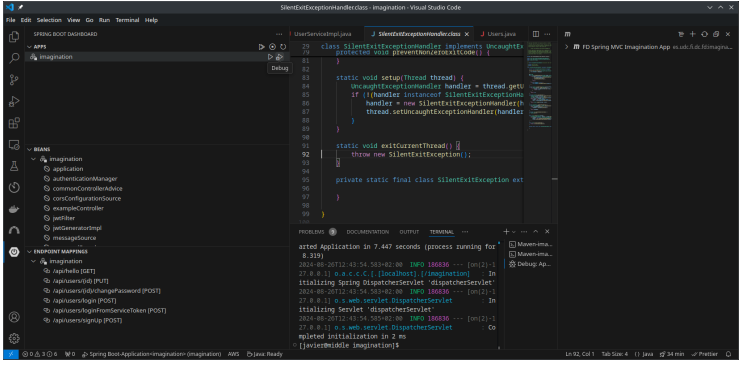
The debugger does not have to run where you are. Three cases you will meet this term:

- **An application server.** VS Code embeds none. You start the server yourself and **attach** — or use the **Spring Boot Dashboard**, or Red Hat's **Community Server Connectors** for Tomcat, Karaf and friends
- **A browser.** Create a `launch.json` for Chrome, point it at your dev server's port, press `F5`: breakpoints in your React code, hit in the browser. Vite's hot module replacement keeps the application's state while you edit
- **A container.** Expose the debug port (`5005`), start the JVM with `-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=*:5005`, and attach from `launch.json` — exactly what we will do in *Lesson 10*

One idea in three disguises: the debugger speaks a protocol to a process. Whether that process is on your laptop, in a container or on another machine changes the configuration, not the technique.

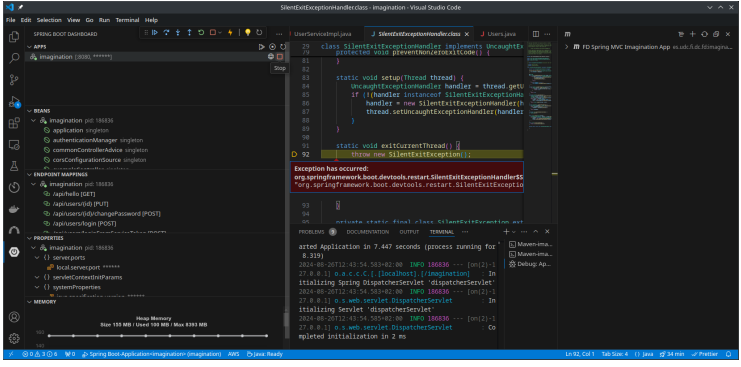
Debugging elsewhere, in practice

Spring Boot Dashboard: the applications of the workspace, run or debug.



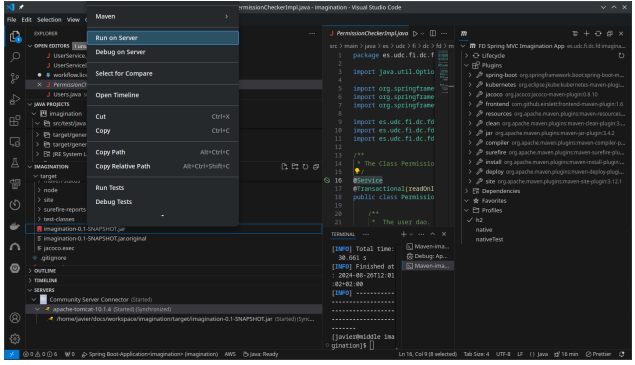
Debugging elsewhere, in practice

The same dashboard, with an application running.



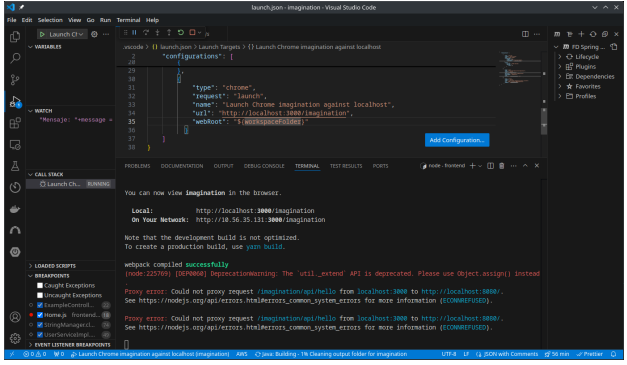
Debugging elsewhere, in practice

Community Server Connectors: start, stop and publish to a server.



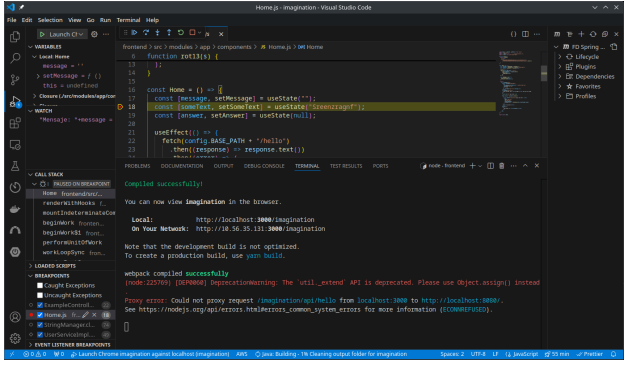
Debugging elsewhere, in practice

A breakpoint in React code, hit from the browser.



Debugging elsewhere, in practice

The same session, stopped in the editor.



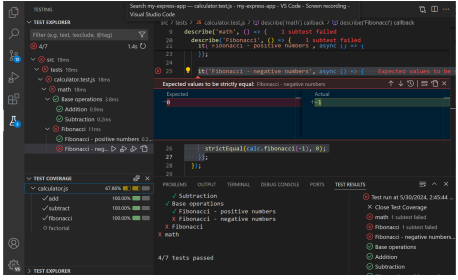
SECTION 6

Testing

6

Tests, from the editor

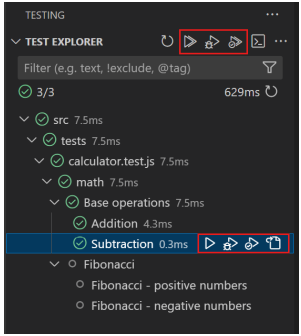
- **Discovery** — the beaker icon in the activity bar, or *Testing: Focus on Test Explorer View*. Your tests appear as a tree, found by the language server, with no configuration
- **Run** — from the Test Explorer, from the gutter next to the test, or from the command palette. One test, one class, everything
- **Debug** — the same run, with breakpoints. This is the single most useful habit in the course: **debug the failing test**, not the application
- **Coverage** — run with coverage and the editor paints which lines ran. *Lesson 9* is where we discuss what that number is worth
- JUnit and TestNG here; Mocha, Jasmine, pytest and others through their own extensions



The Test Explorer

Tests, from the editor

- **Discovery** — the beaker icon in the activity bar, or *Testing: Focus on Test Explorer View*. Your tests appear as a tree, found by the language server, with no configuration
- **Run** — from the Test Explorer, from the gutter next to the test, or from the command palette. One test, one class, everything
- **Debug** — the same run, with breakpoints. This is the single most useful habit in the course: **debug the failing test**, not the application
- **Coverage** — run with coverage and the editor paints which lines ran. *Lesson 9* is where we discuss what that number is worth
- JUnit and TestNG here; Mocha, Jasmine, pytest and others through their own extensions



Run, debug, coverage

A test you can run in one click is a test you will run. That is most of the argument for the whole Test Explorer.

SECTION 7

The team’s editor

7

Your editor configuration belongs in the repository

Six people, six laptops, six sets of preferences — and a 400-line diff because somebody's editor uses tabs. The fix is not a rule in a chat group: it is **four files committed with the code**.

```
my-project/
├── .vscode/
│   ├── settings.json
│   ├── extensions.json
│   ├── launch.json
│   └── tasks.json
├── .editorconfig
├── pom.xml
└ ...
```

- `settings.json` — formatting, indentation, format on save. The diff stops being about whitespace
- `extensions.json` — **recommended** extensions. A new member opens the project and the editor offers exactly what the project needs
- `launch.json` — how to run and debug **this** project. `F5` means the same thing for everybody
- `tasks.json` — the project's commands, one keystroke away
- `.editorconfig` — the same basics for editors that are *not* VS Code

Your editor configuration belongs in the repository

Six people, six laptops, six sets of preferences — and a 400-line diff because somebody's editor uses tabs. The fix is not a rule in a chat group: it is **four files committed with the code**.

```
my-project/
├── .vscode/
│   ├── settings.json
│   ├── extensions.json
│   ├── launch.json
│   └── tasks.json
├── .editorconfig
└── pom.xml  ...
```

- `settings.json` — formatting, indentation, format on save. The diff stops being about whitespace
- `extensions.json` — **recommended** extensions. A new member opens the project and the editor offers exactly what the project needs
- `launch.json` — how to run and debug **this** project. `F5` means the same thing for everybody
- `tasks.json` — the project's commands, one keystroke away
- `.editorconfig` — the same basics for editors that are *not* VS Code

The presentation's message, applied to the editor: the setup must not depend on who sits at the keyboard — and it costs four small files, written once.

What is open source here, and what is not

The source

`github.com/microsoft/vscode` is **MIT**. But the binary Microsoft ships — the one called *Visual Studio Code* — adds branding, telemetry and some proprietary pieces, under its own licence.

VSCodium builds the same source without them. Same editor, different build.

What is open source here, and what is not

The source

`github.com/microsoft/vscode` is **MIT**. But the binary Microsoft ships — the one called *Visual Studio Code* — adds branding, telemetry and some proprietary pieces, under its own licence.

VSCodium builds the same source without them. Same editor, different build.

The marketplace is the catch

The Marketplace terms **restrict its use to Microsoft-branded products**. So VSCodium, Theia and Gitpod use **Open VSX**, the Eclipse Foundation's vendor-neutral registry (2019; a working group since 2023, with Google, Salesforce, Amazon and Siemens).

Some extensions are simply not there.

What is open source here, and what is not

The source

`github.com/microsoft/vscode` is **MIT**. But the binary Microsoft ships — the one called *Visual Studio Code* — adds branding, telemetry and some proprietary pieces, under its own licence.

VSCodium builds the same source without them. Same editor, different build.

The marketplace is the catch

The Marketplace terms **restrict its use to Microsoft-branded products**. So VSCodium, Theia and Gitpod use **Open VSX**, the Eclipse Foundation's vendor-neutral registry (2019; a working group since 2023, with Google, Salesforce, Amazon and Siemens).

Some extensions are simply not there.

And the newest twist: in June 2025 Microsoft open sourced the **GitHub Copilot Chat** extension under MIT, with the stated plan of folding it into the core — while inline completions stayed closed. The AI part of your editor is now partly readable, which is the subject of the next session.

What is open source here, and what is not

The source

`github.com/microsoft/vscode` is **MIT**. But the binary Microsoft ships — the one called *Visual Studio Code* — adds branding, telemetry and some proprietary pieces, under its own licence.

VSCodium builds the same source without them. Same editor, different build.

The marketplace is the catch

The Marketplace terms **restrict its use to Microsoft-branded products**. So VSCodium, Theia and Gitpod use **Open VSX**, the Eclipse Foundation's vendor-neutral registry (2019; a working group since 2023, with Google, Salesforce, Amazon and Siemens).

Some extensions are simply not there.

And the newest twist: in June 2025 Microsoft open sourced the **GitHub Copilot Chat** extension under MIT, with the stated plan of folding it into the core — while inline completions stayed closed. The AI part of your editor is now partly readable, which is the subject of the next session.

Why an engineer should care: "is it open source" is rarely a yes-or-no question. It is a question about which artefact: source, build, extensions, or the service behind them.

The extensions this course expects

Install them when you reach the lesson, not today: a profile full of extensions you do not understand is slower, not better.

EXTENSION	FOR
Extension Pack for Java	everything Java (*today*)
Spring Boot Extension Pack	the framework of the practical
ES7+ React/Redux snippets	the front end
GitLab Workflow	issues, merge requests and pipelines (*Lessons 2, 3, 5*)
SonarLint	static analysis while you type (*Lesson 6*)
Docker	containers, and debugging inside them (*Lesson 10*)
Vitest	front-end tests (*Lesson 9*)
TODO Tree	every `TODO` in the project, in one list

The extensions this course expects

Install them when you reach the lesson, not today: a profile full of extensions you do not understand is slower, not better.

EXTENSION	FOR
Extension Pack for Java	everything Java (*today*)
Spring Boot Extension Pack	the framework of the practical
ES7+ React/Redux snippets	the front end
GitLab Workflow	issues, merge requests and pipelines (*Lessons 2, 3, 5*)
SonarLint	static analysis while you type (*Lesson 6*)
Docker	containers, and debugging inside them (*Lesson 10*)
Vitest	front-end tests (*Lesson 9*)
TODO Tree	every `TODO` in the project, in one list

And put them in `extensions.json` as you go, so the list is the project's, not yours.

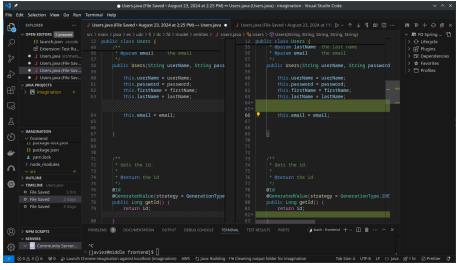
SECTION 8

Version control



Two histories: yours, and the team's

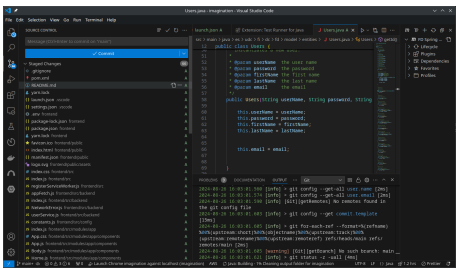
- The **Timeline** at the bottom of the Explorer is local history: every save of a file, kept by the editor, with no repository involved. It is what recovers the version you had twenty minutes ago
- The **Source Control** view is the other one: Git, built in. Changes split into **Changes**, **Staged changes** and **Merge changes**; click any file to see its diff, and edit the unstaged side directly
- The status bar shows the **branch**, whether the tree is dirty, and how many commits you are ahead or behind. Click it to switch branch
- Other systems — Subversion, Mercurial — arrive as extensions



Timeline: local history

Two histories: yours, and the team's

- The **Timeline** at the bottom of the Explorer is local history: every save of a file, kept by the editor, with no repository involved. It is what recovers the version you had twenty minutes ago
- The **Source Control** view is the other one: Git, built in. Changes split into **Changes**, **Staged changes** and **Merge changes**; click any file to see its diff, and edit the unstaged side directly
- The status bar shows the **branch**, whether the tree is dirty, and how many commits you are ahead or behind. Click it to switch branch
- Other systems — Subversion, Mercurial — arrive as extensions



Source Control: the repository

Do not confuse them. The Timeline is a safety net for you, for a few days. Only a commit makes a change exist for anybody else.

GitLab, without leaving the editor

The **GitLab Workflow** extension brings the part of GitLab you use every day into the sidebar — which matters because the alternative is a browser tab, and a browser tab is where the context switch happens:

GitLab, without leaving the editor

The **GitLab Workflow** extension brings the part of GitLab you use every day into the sidebar — which matters because the alternative is a browser tab, and a browser tab is where the context switch happens:

- **Issues and merge requests**: read them, comment, see the changed files, create and review an MR from here (*Lesson 3*)

GitLab, without leaving the editor

The **GitLab Workflow** extension brings the part of GitLab you use every day into the sidebar — which matters because the alternative is a browser tab, and a browser tab is where the context switch happens:

- **Issues and merge requests**: read them, comment, see the changed files, create and review an MR from here (*Lesson 3*)
- **Pipelines**: status of the current branch, open the related MR, retry or cancel a job, and read a job's output in an editor tab (*Lesson 5*)

GitLab, without leaving the editor

The **GitLab Workflow** extension brings the part of GitLab you use every day into the sidebar — which matters because the alternative is a browser tab, and a browser tab is where the context switch happens:

- **Issues and merge requests**: read them, comment, see the changed files, create and review an MR from here (*Lesson 3*)
- **Pipelines**: status of the current branch, open the related MR, retry or cancel a job, and read a job's output in an editor tab (*Lesson 5*)
- **Security findings** on your branch, and **CI/CD validation** of `.gitlab-ci.yml` before you push it — with completion for the CI/CD variables

Validating the pipeline file locally is the one to remember: it turns "push and wait four minutes to discover a typo" into a command.

GitLab, without leaving the editor

The **GitLab Workflow** extension brings the part of GitLab you use every day into the sidebar — which matters because the alternative is a browser tab, and a browser tab is where the context switch happens:

- **Issues and merge requests**: read them, comment, see the changed files, create and review an MR from here (*Lesson 3*)
- **Pipelines**: status of the current branch, open the related MR, retry or cancel a job, and read a job's output in an editor tab (*Lesson 5*)
- **Security findings** on your branch, and **CI/CD validation** of `.gitlab-ci.yml` before you push it — with completion for the CI/CD variables
- **Browse a repository** without cloning it, and manage snippets

Validating the pipeline file locally is the one to remember: it turns "push and wait four minutes to discover a typo" into a command.

The four commands you need this week

Lesson 2 does version control properly — the model, branches, merges, and why any of it works. Until then, this is enough to not lose your work:

COMMAND	WHAT IT DOES
`git clone <url>`	get the project for the first time
`git status`	what has changed, and what is going into the next commit
`git add <file>`	put this change into the next commit
`git commit -m "..."`	record it, with a message somebody else can read
`git pull` / `git push`	bring their work in / send yours out

The four commands you need this week

Lesson 2 does version control properly — the model, branches, merges, and why any of it works. Until then, this is enough to not lose your work:

COMMAND	WHAT IT DOES
`git clone <url>`	get the project for the first time
`git status`	what has changed, and what is going into the next commit
`git add <file>`	put this change into the next commit
`git commit -m "..."`	record it, with a message somebody else can read
`git pull` / `git push`	bring their work in / send yours out

The Source Control view does all of the above with buttons, and you should use it. But learn what each button runs: when something goes wrong — and in a six-person group it will — the fix is always a command, and the error message assumes you know the vocabulary.

The four commands you need this week

Lesson 2 does version control properly — the model, branches, merges, and why any of it works. Until then, this is enough to not lose your work:

COMMAND	WHAT IT DOES
`git clone <url>`	get the project for the first time
`git status`	what has changed, and what is going into the next commit
`git add <file>`	put this change into the next commit
`git commit -m "..."`	record it, with a message somebody else can read
`git pull` / `git push`	bring their work in / send yours out

The Source Control view does all of the above with buttons, and you should use it. But learn what each button runs: when something goes wrong — and in a six-person group it will — the fix is always a command, and the error message assumes you know the vocabulary.

One habit from day one: commit small, commit often, write the message for the person who reads it in March. Usually you.

Your turn

To read

- VS Code documentation — `code.visualstudio.com/docs`
- The Java tutorial — `.../docs/java/java-tutorial`
- B. Johnson, *Visual Studio Code* (2019)

And the keyboard sheet from a few slides ago. Print it, it works.

Exercise, before the next practical: open your group's repository and commit a `.vscode/` folder with `settings.json`, `extensions.json` and a `launch.json` that runs the project. Then ask a teammate to pull it and press F5. If it runs on their machine without a single question in the group chat, you have understood this lesson.

“Debugging is like being a detective in a crime movie where you are also the murderer.”

Filipe Fortes

Integrated Development Environments

Grao en Enxeñaría Informática

@jparapar · javierparapar@udc.es · www.dc.fi.udc.es/~parapar

GEI GRADO EN ENXEÑARÍA INFORMÁTICA

Appendix — Java snippets

PREFIX	DESCRIPTION
`ctor`	public constructor
`dowhile`	do-while statement
`foreach`, `iter`	iterate over an array or Iterable
`fori`	iterate over an array by index
`if`	if statement
`ifelse`	if-else statement
`ifnull`	if, checking for null
`ifnotnull`	if, checking for not null
`main`, `psvm`	public static main method
`new`	create a new object
`switch`	switch statement
`while`	while statement

PREFIX	DESCRIPTION
`private_method`	private method
`private_static_method`	private static method
`prf`	private field
`protected_method`	protected method
`public_method`	public method
`public_static_method`	public static method
`serr`, `syserr`	print to standard error
`sout`, `sysout`	print to standard output
`soutm`, `systrace`	print the current method
`try_catch`	try/catch block
`try_resources`	try-with-resources

Appendix — postfix snippets

Typed **after** an expression: `items.fori`, `name.null`, `value.var`.

SUFFIX	CREATES
`cast`	a cast of the expression
`else`	a negated if statement
`for`	a for statement
`fori`	an indexed for over an array
`forr`	a reverse indexed for
`if`	an if statement

SUFFIX	CREATES
`nnull`	an if, checking for not null
`null`	an if, checking for null
`sysout`	a `System.out.println()` of it
`throw`	throws the given exception
`var`	a new variable holding it
`while`	a while loop

Git Cheat Sheet

Remember: git command --help

Global Git configuration is stored in $HOME/.gitconfig (git config --help)

Create

From existing data
```
cd ~/projects/myproject
git init
git add .
```

From existing repo
```
git clone ~/existing/repo ~/new/repo
git clone git://host.org/project.git
git clone ssh://you@host.org/proj.git
```

Show

Files changed in working directory
```
git status
```

Changes to tracked files
```
git diff
```

What changed between $ID1 and $ID2
```
git diff $id1  $id2
```

History of changes
```
git log
```

History of changes for file with diffs
```
git log -p $file $dir/ec/tory/
```

Who changed what and when in a file
```
git blame $file
```

A commit identified by $ID
```
git show $id
```

A specific file from a specific $ID
```
git show $id:$file
```

All local branches
```
git branch
```
(star '*' marks the current branch)

Cheat Sheet Notation

$id : notation used in this sheet to represent either a commit id, branch or a tag name
$file : arbitrary file name
$branch : arbitrary branch name

Concepts

Git Basics

```
master    : default development branch
origin    : default upstream repository
HEAD      : current branch
HEAD^     : parent of HEAD
HEAD~4    : the great-great grandparent of HEAD
```

Revert

Return to the last committed state
```
git reset --hard
```
⚠ you cannot undo a hard reset

Revert the last commit
```
git revert HEAD
```
Creates a new commit

Revert specific commit
```
git revert $id
```
Creates a new commit

Fix the last commit
```
git commit -a --amend
```
(after editing the broken files)

Checkout the $id version of a file
```
git checkout $id $file
```

Branch

Switch to the $id branch
```
git checkout $id
```

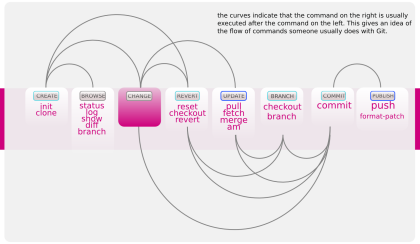
Merge branch1 into branch2
```
git checkout $branch2
git merge branch1
```

Create branch named $branch based on the HEAD
```
git branch $branch
```

Create branch $new_branch based on branch $other and switch to it
```
git checkout -b $new_branch $other
```

Delete branch $branch
```
git branch -d $branch
```

Commands Sequence



Update

Fetch latest changes from origin
```
git fetch
```
(but this does not merge them).

Pull latest changes from origin
```
git pull
```
(does a fetch followed by a merge)

Apply a patch that some sent you
```
git am -3 patch.mbox
```
(in case of a conflict, resolve and use `git am --resolved`)

Publish

Commit all your local changes
```
git commit -a
```

Prepare a patch for other developers
```
git format-patch origin
```

Push changes to origin
```
git push
```

Mark a version / milestone
```
git tag v1.0
```

Useful Commands

Finding regressions
```
git bisect start      (to start)
git bisect good $id   ($id is the last working version)
git bisect bad $id    ($id is a broken version)

git bisect bad/good   (to mark it as bad or good)
git bisect visualize  (to launch gitk and mark it)
git bisect reset      (once you're done)
```

Check for errors and cleanup repository
```
git fsck
git gc --prune
```

Search working directory for foo()
```
git grep "foo()"
```

Resolve Merge Conflicts

To view the merge conclicts
```
git diff      (complete conflict diff)
git diff --base  $file  (against base file)
git diff --ours  $file  (against your changes)
git diff --theirs $file  (against other changes)
```

To discard conflicting patch
```
git reset --hard
git rebase --skip
```

After resolving conflicts, merge with
```
git add $conflicting_file  (do for all resolved files)
git rebase --continue
```

Zack Rusin
Based on the work of
Jason Harris and
Xpirinx Cetin.